
Table of Contents

.....	1
Load Velocity Measurements	1
Define Physical Constants	1
Continuous-Time State-Space Model	1
Discretization using Zero-Order Hold (ZOH)	2
Kalman Filter Parameters	2
Kalman Filter Implementation	2
Trajectory Comparison (Position Estimates)	3
Velocity Comparison – X Direction	3
Velocity Comparison – Y Direction	4
Position Evolution Over Time	5

```
clc;
clear;
close all;
```

Load Velocity Measurements

```
data = load('HW1_Q1_Velocity_Measurements.txt');

t = data(:,1);
vx_meas = data(:,2);
vy_meas = data(:,3);

N = length(t);
dt = t(2) - t(1); % Sampling time (0.01 s)
```

Define Physical Constants

```
g = 9.81; % Gravity (m/s^2)
```

Continuous-Time State-Space Model

State vector: $X = [ux \ vx \ uy \ vy]'$

```
Ac = [ 0  1  0  0;
       0  0  0  0;
       0  0  0  1;
       0  0  0  0];
```

```
Bc = [ 0;
       0;
       0;
      -g];
```

```
Cc = [ 0  1  0  0;
       0  0  0  1]; % velocity measurements only
```

```
Dc = zeros(2,1);
```

```
sysc = ss(Ac,Bc,Cc,Dc);
```

Discretization using Zero-Order Hold (ZOH)

```
sysd = c2d(sysc,dt,"zoh");
```

```
Ad = sysd.A;
```

```
Bd = sysd.B;
```

```
Hd = sysd.C;
```

Kalman Filter Parameters

```
% Measurement noise covariance
```

```
R = [4 0;  
     0 16];
```

```
Q = zeros(4);          % No process noise
```

```
% Initial state (same as part a)
```

```
X0 = [300; 100; 200; 60];
```

```
% Two covariance cases
```

```
P_cases = {eye(4)*1e4, eye(4)*1e8};
```

Kalman Filter Implementation

```
estimates = cell(2,1);
```

```
for caseID = 1:2
```

```
    P = P_cases{caseID};
```

```
    X = X0;
```

```
    X_est = zeros(4,N);
```

```
    for k = 1:N
```

```
        % Prediction
```

```
        X_pred = Ad*X + Bd;
```

```
        P_pred = Ad*P*Ad' + Q;
```

```
        % Measurement vector (velocity only)
```

```
        z = [vx_meas(k); vy_meas(k)];
```

```
        % Kalman Gain
```

```
        K = P_pred*Hd'/(Hd*P_pred*Hd' + R);
```

```
        % Update
```

```
        X = X_pred + K*(z - Hd*X_pred);
```

```

P = (eye(4) - K*Hd)*P_pred;

X_est(:,k) = X;

end

estimates{caseID} = X_est;

end

```

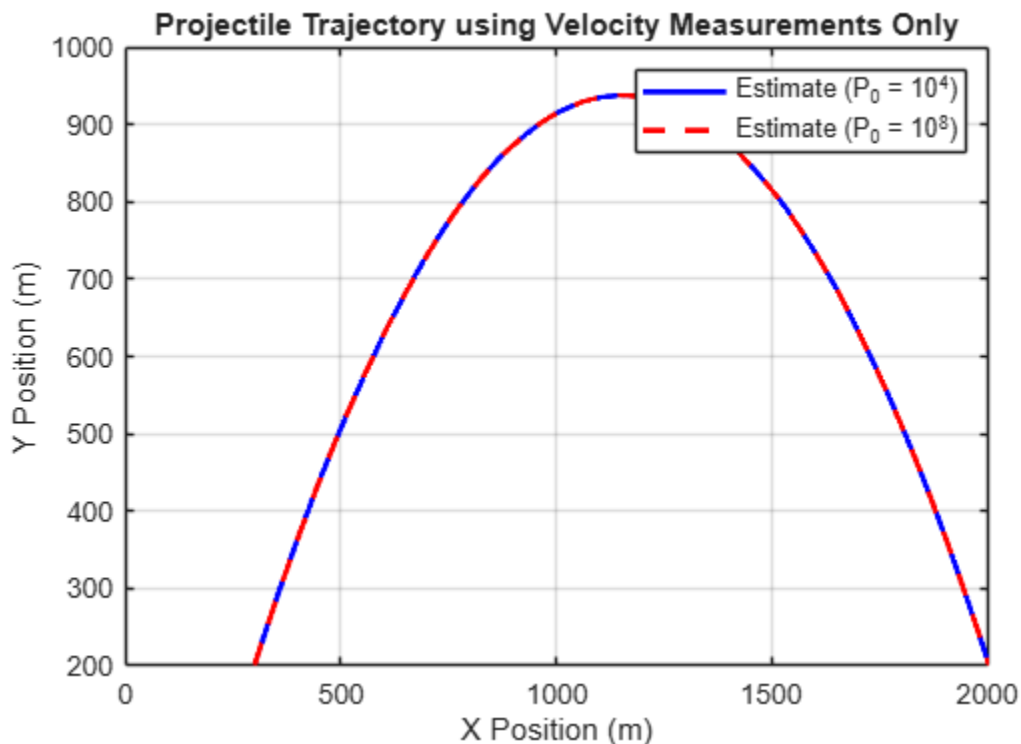
Trajectory Comparison (Position Estimates)

```

figure;
plot(estimates{1}(1,:), estimates{1}(3,:), 'b', 'LineWidth',2); hold on;
plot(estimates{2}(1,:), estimates{2}(3,:), 'r--', 'LineWidth',2);

xlabel('X Position (m)');
ylabel('Y Position (m)');
legend('Estimate (P_0 = 10^4)', 'Estimate (P_0 = 10^8)');
title('Projectile Trajectory using Velocity Measurements Only');
grid on;

```



Velocity Comparison – X Direction

```

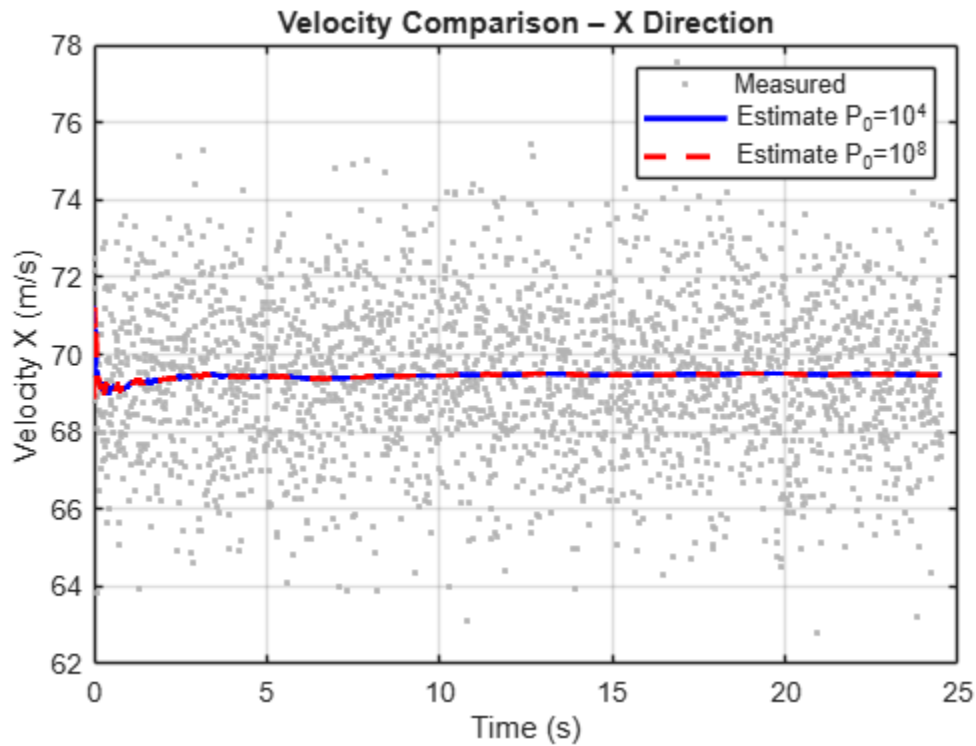
figure;
plot(t, vx_meas, '.', 'Color',[0.7 0.7 0.7]); hold on;
plot(t, estimates{1}(2,:), 'b', 'LineWidth',2);
plot(t, estimates{2}(2,:), 'r--', 'LineWidth',2);

```

```

legend('Measured', 'Estimate P_0=10^4', 'Estimate P_0=10^8');
xlabel('Time (s)');
ylabel('Velocity X (m/s)');
title('Velocity Comparison - X Direction');
grid on;

```



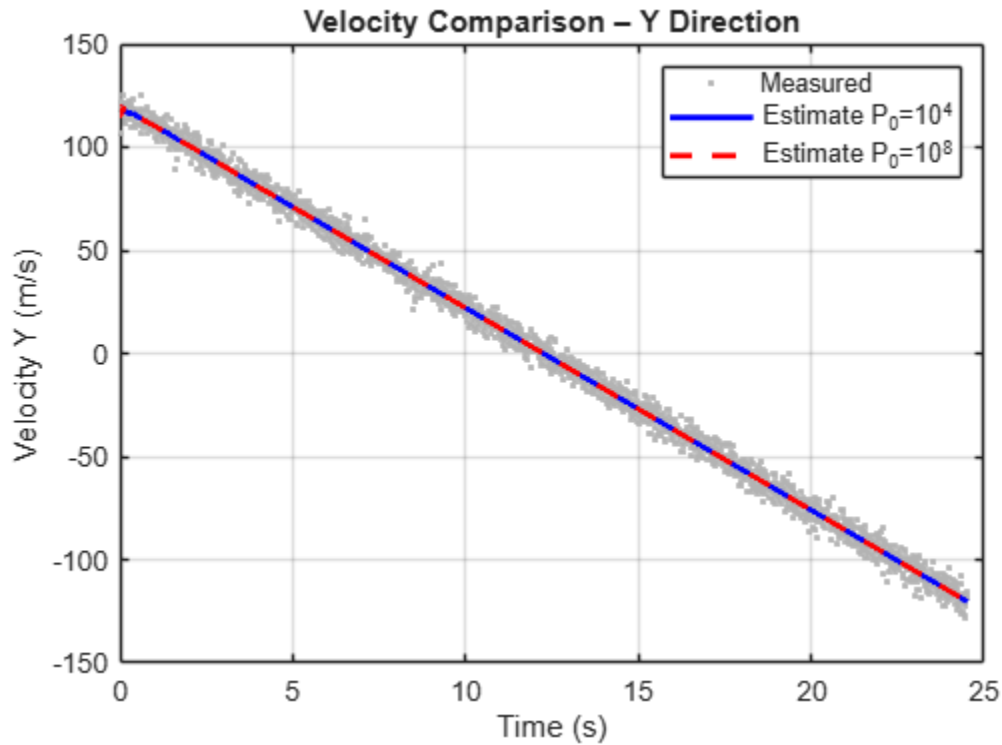
Velocity Comparison – Y Direction

```

figure;
plot(t, vy_meas, '.', 'Color',[0.7 0.7 0.7]); hold on;
plot(t, estimates{1}(4,:), 'b', 'LineWidth',2);
plot(t, estimates{2}(4,:), 'r--', 'LineWidth',2);

legend('Measured', 'Estimate P_0=10^4', 'Estimate P_0=10^8');
xlabel('Time (s)');
ylabel('Velocity Y (m/s)');
title('Velocity Comparison - Y Direction');
grid on;

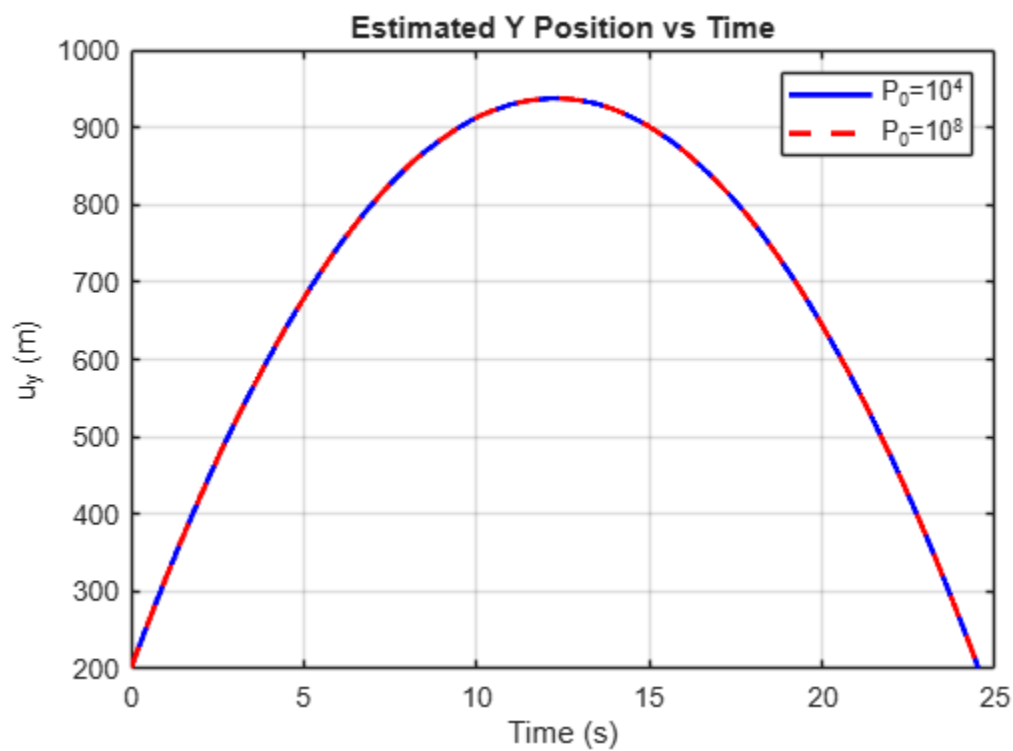
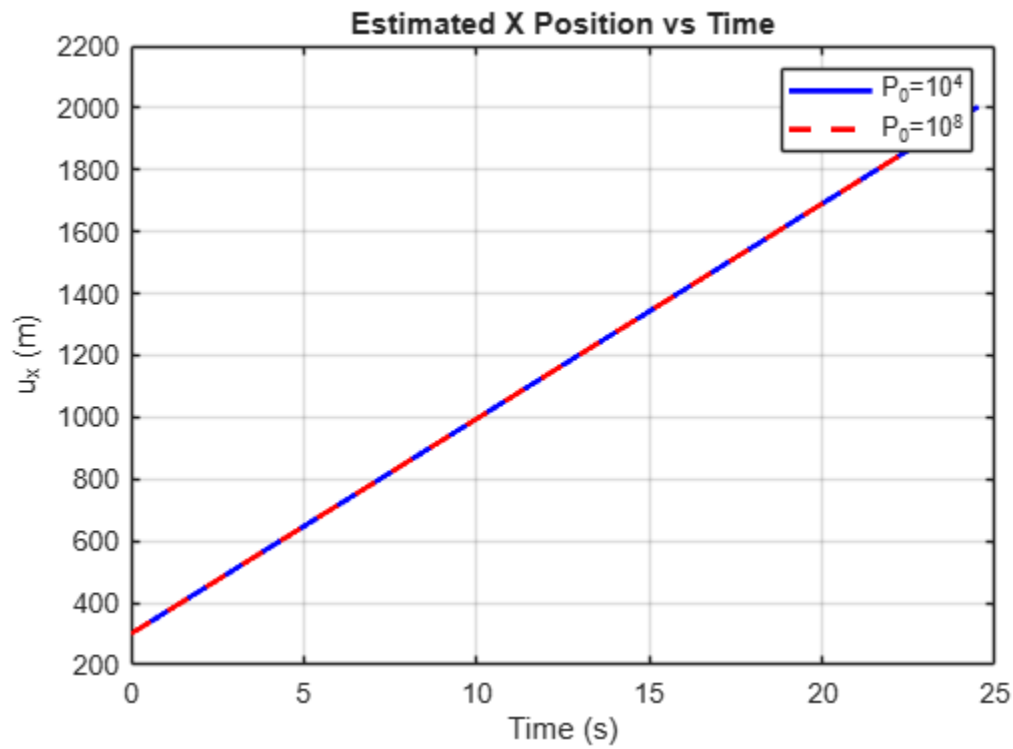
```



Position Evolution Over Time

```
figure;  
plot(t, estimates{1}(1,:), 'b', 'LineWidth', 2); hold on;  
plot(t, estimates{2}(1,:), 'r--', 'LineWidth', 2);  
xlabel('Time (s)');  
ylabel('u_x (m)');  
legend('P_0=10^4', 'P_0=10^8');  
title('Estimated X Position vs Time');  
grid on;
```

```
figure;  
plot(t, estimates{1}(3,:), 'b', 'LineWidth', 2); hold on;  
plot(t, estimates{2}(3,:), 'r--', 'LineWidth', 2);  
xlabel('Time (s)');  
ylabel('u_y (m)');  
legend('P_0=10^4', 'P_0=10^8');  
title('Estimated Y Position vs Time');  
grid on;
```



Published with MATLAB® R2025b